

Applied Bayesian Statistics in Animal & Biological Sciences

5) Discrete data modeling

Henk J. van Lingen

hvanling@uoguelph.ca

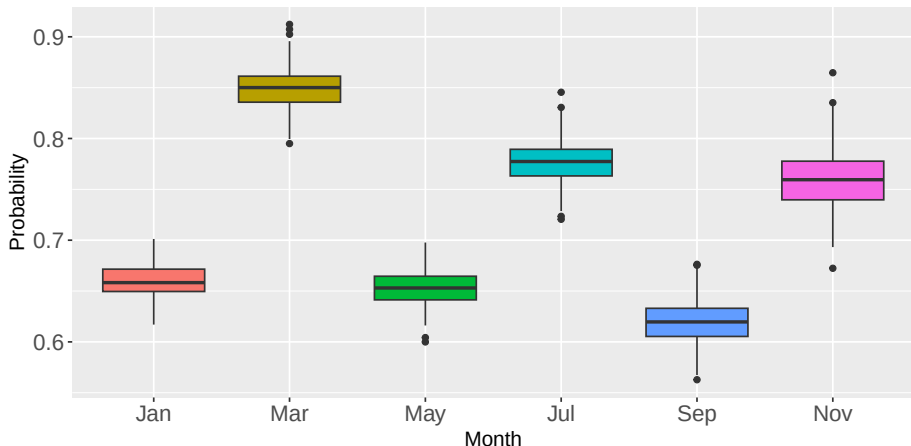
May 14, 2026



Today's program

- Dataset for chicken vaccination rates against Newcastle disease in rural Tanzania (binomial model + logit-function)
- Traffic incident count data (poisson models, linear prediction, log-link function, over-dispersion)
- Rural Tanzania chicken numbers per household after vaccination dataset (poisson models, over-dispersion and hierarchical modeling)
- The negative binomial distribution for over-dispersed data

Chicken vaccination rates in Tanzania by campaign month



- Binomial modeling: Does month of vaccination campaign affect the relative participation rate?

Binary data modeling of vaccination rates in rural Tanzania

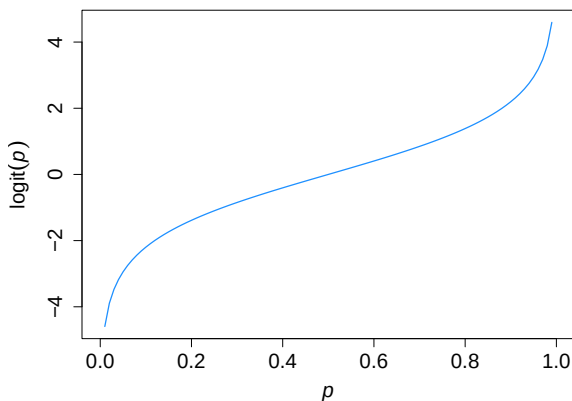
	Jan	March	May	July	Sept	Nov
n	709	301	804	400	674	198
y	468	256	525	310	418	151
Odds ratio	0.66	0.85	0.65	0.78	0.62	0.76

$$y_i \sim \text{Binomial}(n, p) \quad (1)$$

- n trials, and p probability of success (the odd ratios)
- Binary data: 468 times a 1 and $709-468=241$ times a 0 in January

De Bruyn et al., (2017)

Linear prediction and binary data (binomial) modeling



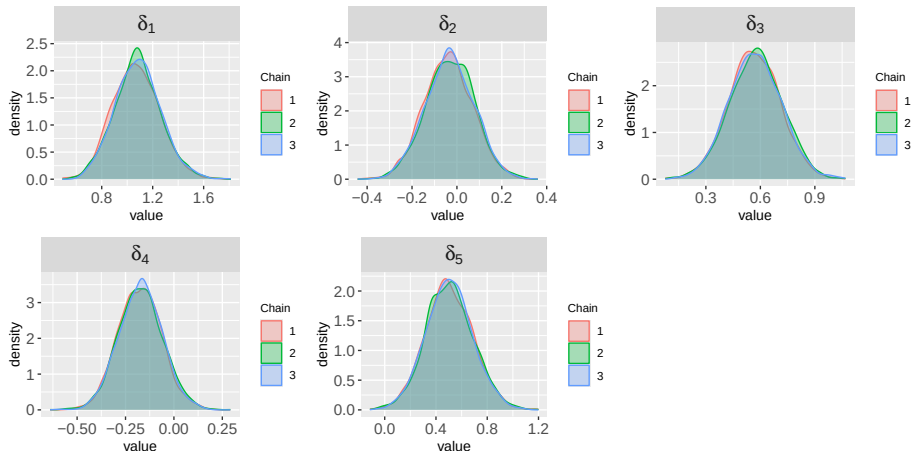
$$\text{logit}(p_i) = \log\left(\frac{p_i}{1-p_i}\right) = \eta_i = \alpha + \beta_{\text{month},i} \quad (2)$$

- The logit function connects (binomial) probabilities ($0 \leq P \leq 1$) to linear predictors $[-\infty, \infty]$.

Logit binomial prediction of vaccination rates in RSTAN

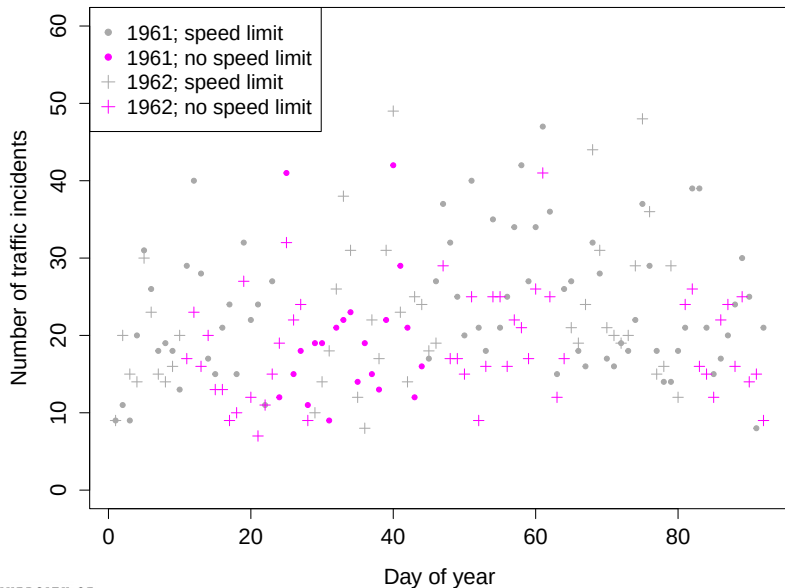
```
binom_chicken <- "  
data {  
  int<lower=1> N;           // number of months  
  int<lower=0> y[N];         // successes  
  int<lower=0> n[N];         // trials  
  int<lower=1,upper=N> month[N];  
}  
  
parameters {  
  real alpha;               // baseline (month 1)  
  vector[N-1] delta;        // differences vs baseline  
}  
  
transformed parameters {  
  vector[N] beta;  
  beta[1] = 0;               // reference month  
  beta[2:N] = delta;  
}  
  
model {  
  alpha ~ normal(0, 2); delta ~ normal(0, 2); // priors  
  
  // likelihood  
  for (i in 1:N) {  
    y[i] ~ binomial_logit(n[i], alpha + beta[month[i]]);  
  }  
}  
"
```

Logit binomial vaccination rate effects relative to January

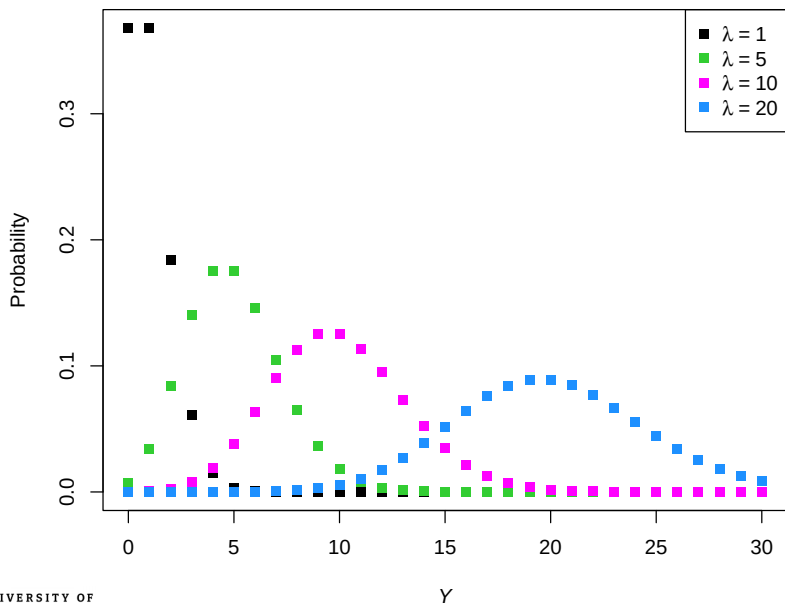


- Vaccination rates in March, July and Nov are higher than in January
- Probability distributions of pairwise comparisons obtained solidly ... ?

Swedish traffic incidents in 1961-62; (?) speed limit applied



Poisson distribution with mean and variance parameter λ



Discrete/Count data modeling using poisson distributions

Poisson is a special case of the Negative-Binomial distribution

$$P(Y = k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad (3)$$

- For discrete values $k = \{0, 1, 2, \dots\}$
- λ is a continuous parameter
- $y_i \sim \text{NegBinomial}(\mu_i, \phi); \text{var}(y_i) = \mu_i + \mu_i^2/\phi$

Discrete/Count data modeling using poisson distributions

Poisson is a special case of the Negative-Binomial distribution

$$P(Y = k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad (3)$$

- For discrete values $k = \{0, 1, 2, \dots\}$
- λ is a continuous parameter
- $y_i \sim \text{NegBinomial}(\mu_i, \phi); \text{var}(y_i) = \mu_i + \mu_i^2/\phi$

Fit traffic incident counts: $y \sim \text{Poisson}(\lambda = \dots)$

Simple poisson model coding in RSTAN

```
library(rstan)
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)

pois_simple <- "
  data {
    int<lower=0> N;
    int<lower=0> y[N];
    real prior_lambda;
  }
  parameters {
    real<lower=0> lambda;
  }
  model {
    lambda ~ uniform(0, prior_lambda);
    y ~ poisson(lambda);
  }
"

# Initialize STAN model
pois_simple_mod <- stan_model(model_code=pois_simple, model_name="pois_simple")

# Assign data object and sample from Poisson model
data(Traffic, package = "MASS") # Load traffic data from MASS package
Traffic_dat01 <- list(N=nrow(Traffic), y=Traffic$y, prior_lambda=20) # Assign traffic data to a list
Traffic_dat02 <- list(N=nrow(Traffic), y=Traffic$y, prior_lambda=50) # Adjust prior
```

Poisson model output from RSTAN

```
pois_simple.rs1 <- sampling(pois_simple_mod, data=Traffic_dat01, iter=10e3, chains=4, thin=10)
pois_simple.rs2 <- sampling(pois_simple_mod, data=Traffic_dat02, iter=10e3, chains=4, thin=10)
```

```
write.csv(print(summary(pois_simple.rs1, pars=c("lambda", "lp__"))$summary),
           "pois_simple_rs01.csv")
```

##	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
## lambda	19.930	0.002016	0.06625	19.754	19.897	19.952	19.979	19.998	1079.63	0.99982

```
write.csv(print(summary(pois_simple.rs2, pars=c("lambda", "lp__"))$summary),
           "pois_simple_rs02.csv")
```

##	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
## lambda	21.540	0.009663	0.34206	20.879	21.304	21.528	21.760	22.239	1282.77	1.00035

Generalized linear prediction and the log-link function

Linear prediction

$$y \sim \text{Poisson}(\eta = \mathbf{X}\beta)$$

Generalized linear prediction and the log-link function

Linear prediction

$$y \sim \text{Poisson}(\eta = \mathbf{X}\beta)$$

Log-link function

- Instead of "raw linear prediction", use a function to avoid negative values:

$$y \sim \text{Poisson}(\exp(\mathbf{X}\beta))$$

$$\text{or: } \log(E[y]) = \mathbf{X}\beta$$

Log-link linear prediction model coding in STAN

```
pois_linpred <- "  
data {  
  int<lower=1> N; int<lower=1> K;  
  int<lower=0> y[N];  
  matrix[N,K] x;  
  real<lower=0> prior_sd;  
}  
  
parameters {  
  real alpha;  
  vector[K] beta;  
}  
  
transformed parameters {  
  vector[N] eta;  
  eta = alpha + x * beta; // linear predictor  
}  
  
model {  
  // priors  
  alpha ~ normal(3, prior_sd);  
  beta ~ normal(0, prior_sd);  
  
  y ~ poisson_log(eta); // log link  
}  
"  
  
pois_linpred_mod <- stan_model(model_code=pois_linpred, model_name="pois_linpred")
```


Generalized linear Poisson model and over-dispersion

Variance in the counts exceeds the expected value of a Poisson

- Over-dispersion model with latent variable l and residual e_{res} :

$$\text{For: } \log(E[y]) = \mathbf{X}\beta$$

use: $y \sim \text{Poisson}(\exp(l = \eta + e_{\text{res}}))$ with $e_{\text{res}} \sim N(0, \sigma_e)$

Generalized linear Poisson model and over-dispersion

Variance in the counts exceeds the expected value of a Poisson

- Over-dispersion model with latent variable l and residual e_{res} :

$$\text{For: } \log(E[y]) = \mathbf{X}\beta$$

$$\text{use: } y \sim \text{Poisson}(\exp(l = \eta + e_{\text{res}})) \text{ with } e_{\text{res}} \sim N(0, \sigma_e)$$

(Non)-centered parametrization: e_{res} and σ_e strongly (in)dependent

$$\frac{e_{\text{res}}}{\sigma_e} = e_{\text{raw}} \sim N(0, 1), \quad e_{\text{res}} = \sigma_e \cdot e_{\text{raw}} \quad (4)$$

- e_{raw} is independent of σ_e (prevents sampling issues)

Over-dispersed log-link linear prediction model coding

```
pois_linpred_disp <- "
data {
  int<lower=1> N; int<lower=1> K;
  int<lower=0> y[N]; matrix[N,K] x;
  real<lower=0> prior_sd;
}

parameters {
  real alpha; vector[K] beta;
  vector[N] e_raw; real<lower=0> sigma_e;
}

transformed parameters {
  // linear predictor and residual and latent variable
  vector[N] e_res; vector[N] eta; vector[N] latent;

  e_res = sigma_e * e_raw; eta = alpha + x * beta;
  latent = eta + e_res;
}

model {
  alpha ~ normal(3, prior_sd); beta ~ normal(0, prior_sd); // normal priors
  e_raw ~ normal(0, 1); sigma_e ~ normal(0, 0.1); // (half)-normal priors

  y ~ poisson_log(latent); // log link function
}
"

pois_linpred_disp_mod <- stan_model(model_code=pois_linpred_disp, model name="pois_linpred_disp")
```

Linear prediction using (over-dispersed) model outputs

```
# Initialize data and assign new data list object
```

```
data(Traffic, package = "MASS")
```

```
Traffic_dat_list <- list(N=nrow(Traffic), K=3, y=Traffic[,4], x=Traffic[,4])
```

```
Traffic_dat_list$x$limit <- as.integer(Traffic_dat_list$x$limit)-1
```

```
Traffic_dat_list$x$year <- as.integer(Traffic_dat_list$x$year)
```

```
# Multiple generalized linear regression without dispersion
```

```
Traffic_dat_list$prior_sd <- 1 # prior SD for alpha and beta
```

```
pois_linpred.rs1 <- sampling(pois_linpred_mod, data=Traffic_dat_list, iter=10e3, chains=4, thin=10)
```

```
pois_linpred_d.rs1 <- sampling(pois_linpred_disp_mod, data=Traffic_dat_list, iter=10e3, chains=4, thin=10)
```

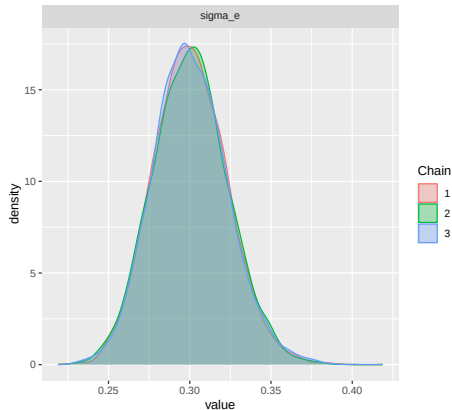
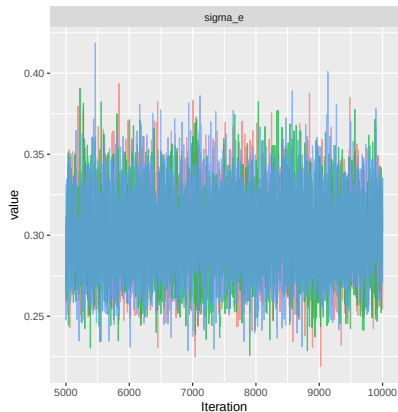
```
print(summary(pois_linpred.rs1, pars=c("alpha", "beta", "lp__", "eta"))$summary)
```

#	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
# alpha	3.0987	0.0013	0.0575	2.9856	3.0595	3.0993	3.1369	3.2126	1890.763	1.0003
# beta[1]	-0.0570	0.0007	0.0331	-0.1213	-0.0793	-0.0574	-0.0345	0.0093	1953.237	1.0000
# beta[2]	0.0025	0.0000	0.0006	0.0013	0.0020	0.0025	0.0029	0.0037	3614.539	0.9998
# beta[3]	-0.1748	0.0008	0.0357	-0.2436	-0.1990	-0.1754	-0.1504	-0.1044	2234.596	0.9998
# eta[1]	3.0442	0.0007	0.0373	2.9720	3.0188	3.0437	3.0697	3.1185	2534.137	1.0003

```
print(summary(pois_linpred_d.rs1, pars=c("alpha", "beta", "sigma_e", "lp__", "eta"))$summary)
```

#	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
# alpha	3.0587	0.0016	0.0997	2.8626	2.9922	3.0588	3.1264	3.2542	4046.491	1.0004
# beta[1]	-0.0653	0.0009	0.0575	-0.1794	-0.1038	-0.0658	-0.0272	0.0474	3761.128	1.0003
# beta[2]	0.0026	0.0000	0.0011	0.0005	0.0019	0.0025	0.0033	0.0046	5540.265	1.0007
# beta[3]	-0.1708	0.0009	0.0602	-0.2880	-0.2105	-0.1721	-0.1311	-0.0520	4885.349	1.0004
# sigma_e	0.3005	0.0003	0.0226	0.2584	0.2849	0.2995	0.3151	0.3483	5636.587	0.9997
# eta[1]	2.9960	0.0009	0.0637	2.8692	2.9533	2.9955	3.0395	3.1202	5060.194	1.0006
# eta[2]	2.9985	0.0009	0.0629	2.8734	2.9565	2.9981	3.0415	3.1211	5054.191	1.0006

Linear prediction (overdispersed) model outputs



- Over-dispersion: σ_e has shifted away from zero, whereas $\sigma_e = 0$ implies the response conforms to a standard Poisson

Generate posterior predictive distributions in RSTAN?

Finish the STAN code for the models with and without overdispersion parameter:

```
ypred[n] = poisson_log_rng(... ..);
```

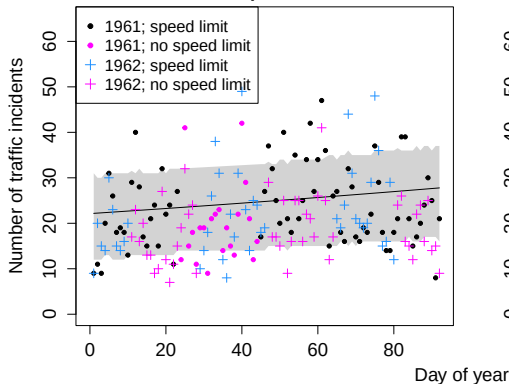
Poisson posterior predictive distributions - STAN code

```
poisson_linpred <- "  
  ...  
  
generated quantities {  
  int ypred[N]; vector[N] eta_rs;  
  eta_rs = alpha + x[,2] * beta[2];  
  
  for (n in 1:N){  
    ypred[n] = poisson_log_rng(eta_rs[n]);  
  } // End for loop n  
}  
"
```

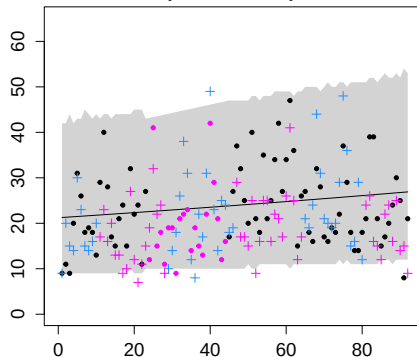
```
poisson_linpred_disp <- "  
  ...  
  
generated quantities {  
  int ypred[N]; real pred_res; vector[N] eta_rs;  
  eta_rs = alpha + x[,2] * beta[2];  
  
  for (n in 1:N){  
    pred_res = normal_rng(0, sigma_e);  
    ypred[n] = poisson_log_rng(fmin(eta_rs[n],20) + pred_res);  
  } // End for loop n  
}  
"
```

Posterior predictive distributions of Poisson models

Linear prediction



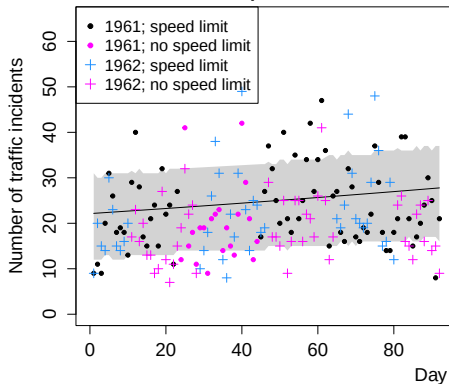
Over-dispersed linear prediction



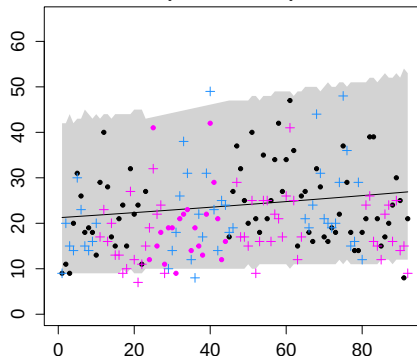
- Over-dispersion model obviously fits the data the best

Posterior predictive distributions of Poisson models

Linear prediction

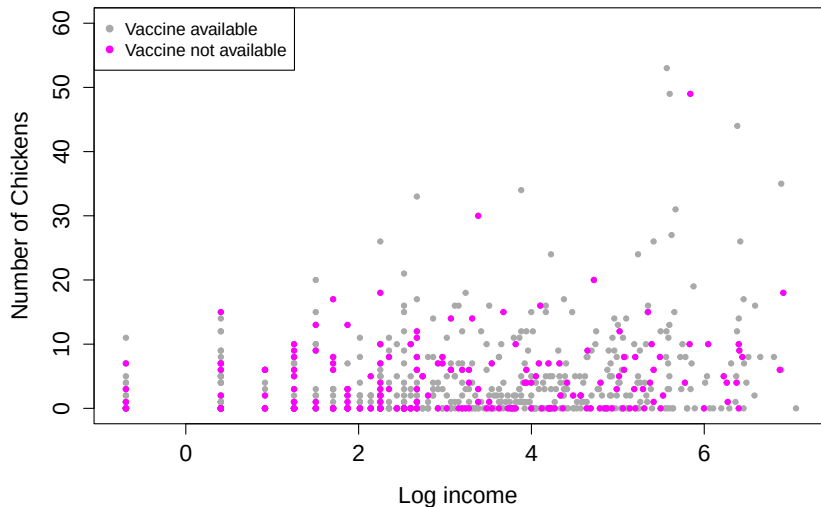


Over-dispersed linear prediction



- Over-dispersion model obviously fits the data the best
- Elaborate traffic incident modeling in *MCMCglmm* course notes

Chickens per household after vaccination campaigning



- Poisson modeling: Do vaccine availability and income predict the number of chickens at the end of the vaccination campaign?

Poisson modeling of chicken count data

Generalized linear prediction

$$y_i \sim \text{Poisson}(\exp(\eta_i + \epsilon_i)); \quad \epsilon_i \sim N(0, \sigma_e) \quad (5)$$

$$\eta_i = \alpha_{\nu,i} + \beta x_i \quad \text{for vaccination effect } \alpha_{\nu} \text{ and income predictor } \beta \quad (6)$$

Poisson modeling of chicken count data

Generalized linear prediction

$$y_i \sim \text{Poisson}(\exp(\eta_i + \epsilon_i)); \quad \epsilon_i \sim N(0, \sigma_e) \quad (5)$$

$$\eta_i = \alpha_{\nu,i} + \beta x_i \quad \text{for vaccination effect } \alpha_{\nu} \text{ and income predictor } \beta \quad (6)$$

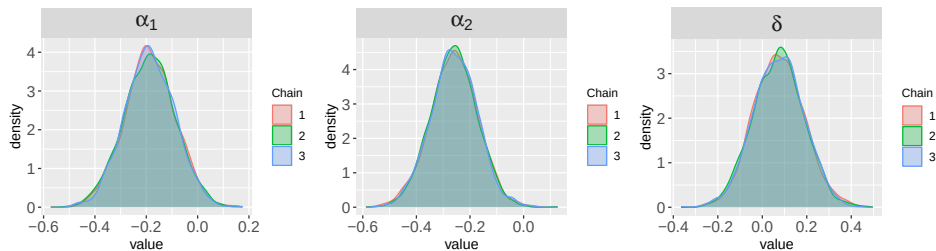
Generalized linear hierarchical prediction - include village effects

$$y_{ij} \sim \text{Poisson}(\exp(\eta_{ij} + u_j + \epsilon_{ij})); \quad u_j \sim N(0, \sigma_u) \quad (7)$$

Poisson log-linear prediction of chicken counts in RSTAN

```
poisson_linpred_disp <- "  
data {  
  int<lower=1> N; int<lower=0> y[N]; vector[N] x; // data points N  
  int<lower=1> M; int<lower=1,upper=M> v[N];      // vacc available as 1's and 2's in v  
  real<lower=0> prior_sd; real prior_alpha; real prior_beta; // prior parameters  
}  
  
parameters {  
  vector[M] alpha; real beta;  
  vector[N] e_raw;  real<lower=0> sigma_e;  
}  
  
transformed parameters {  
  // linear predictor and residual and latent variable  
  vector[N] e_res; vector[N] eta; vector[N] latent;  
  
  e_res = sigma_e * e_raw; eta = alpha[v] + x * beta;  
  latent = eta + e_res;  
}  
  
model {  
  alpha ~ normal(prior_alpha, prior_sd);  
  beta ~ normal(prior_beta, prior_sd);  
  e_raw ~ normal(0, 1); sigma_e ~ normal(0, 2); // half-normal  
  
  y ~ poisson_log(latent); // log link  
}  
"
```

Poisson log-linear vaccination effect on chicken counts

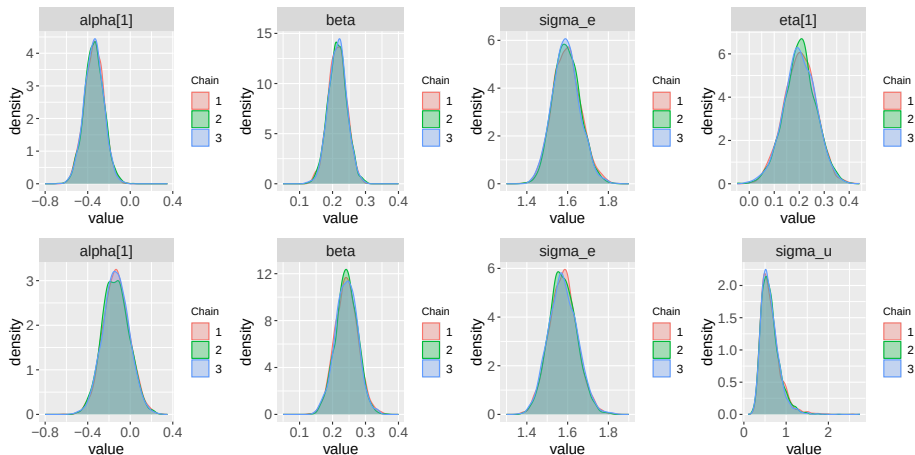


- Credible interval of δ , which is $\alpha_1 - \alpha_2$, contains zero
- No vaccination effect on number of chickens

Poisson hierarchical prediction of chicken counts in RSTAN

```
poisson_linpred_disp_hier <- "  
data {  
  int<lower=1> N; int<lower=0> y[N]; vector[N] x; // data points N  
  int<lower=1> M; int<lower=1,upper=M> v[N]; // vacc available as 1's and 2's in v  
  int<lower=1> K; int<lower=1,upper=K> vill[N]; // villages K  
  real<lower=0> prior_sd; real prior_alpha; real prior_beta;  
}  
  
parameters {  
  vector[M] alpha; real beta;  
  vector[N] e_raw; vector[K] u;  
  real<lower=0> sigma_u; real<lower=0> sigma_e;  
}  
  
transformed parameters {  
  // linear predictor and residual and latent variable  
  vector[N] eta; vector[N] e_res; vector[N] latent;  
  
  e_res = sigma_e * e_raw; eta = alpha[v] + x * beta + u[vill];  
  latent = eta + e_res;  
}  
  
model {  
  alpha ~ normal(prior_alpha, prior_sd); beta ~ normal(prior_beta, prior_sd);  
  e_raw ~ normal(0, 1); u ~ normal(0, sigma_u);  
  sigma_u ~ cauchy(0, 0.5); sigma_e ~ cauchy(0, 0.5); // half-cauchy  
  
  y ~ poisson_log(latent); // log-link  
}"
```

Poisson (hierarchical) model for over-dispersed counts



- $\beta > 0$, income increases the number of chickens per household
- σ_u hardly shifted away from zero, minor village effect?

Generate posterior predictive distributions in RSTAN?

Finish the STAN code:

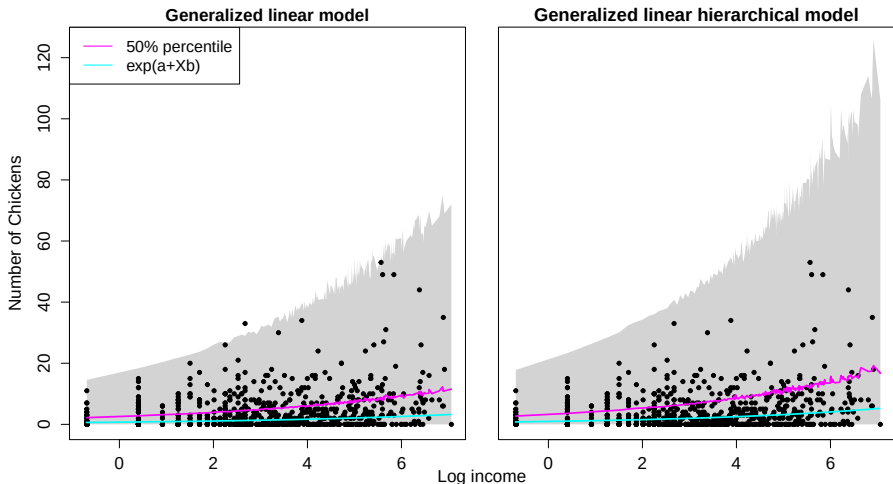
```
ypred[n] = poisson_log_rng(... ..);
```

Generate posterior predictive distributions in RSTAN!

```
...  
generated quantities {  
  int ypred[N];  
  vector[N] eta_rs;  
  
  eta_rs = alpha[v] + x * beta;  
  
  for (n in 1:N){  
    real pred_res = normal_rng(0, sigma_e);  
    ypred[n] = poisson_log_rng(fmin(eta_rs[n] + pred_res, 20)); // log(lambda) <= 20  
  } // End for loop n  
}"
```

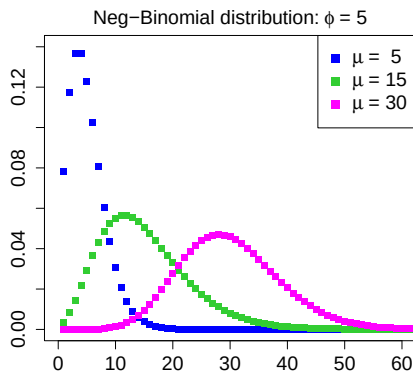
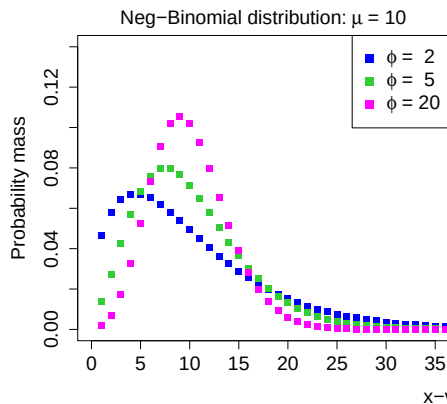
```
...  
generated quantities {  
  int ypred[N];  
  vector[N] eta_rs;  
  
  eta_rs = alpha[v] + x * beta;  
  
  for (n in 1:N){  
    real pred_u = normal_rng(0, sigma_u);  
    real pred_res = normal_rng(0, sigma_e);  
    ypred[n] = poisson_log_rng(fmin(eta_rs[n] + pred_u + pred_res, 20)); // log(lambda) <= 20  
  } // End for loop n  
}"
```

Log-link Poisson posterior predictive distribution



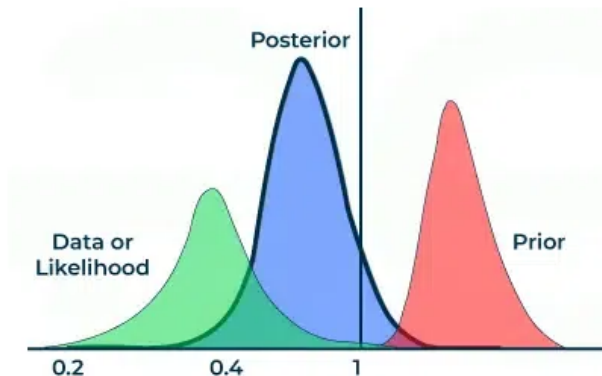
- Hierarchical model has larger noise/over-dispersion non-linearity
- Hierarchical model appears over-parametrized

General case: Negative Binomial distribution



- Over-dispersion is assumed by definition
- Commonly used distribution in genomics and transcriptomics read count analysis

Thank you for your attention!



Questions?

Exercise lecture 5

- Consider the litter size data sets: a) parity effects in pigs, b) breed effects in pigs, c) breeds effects in dogs
- Fit the various poisson models to these datasets
- Consider various priors and generate posterior predictive distributions
- Evaluate breed and parity effects and over-dispersion
- Recode the overdispersed-poisson model as a negative-binomial, fit to one of the 3 datasets and the Traffic incident data, and evaluate the fits (use the posterior (predictive) distributions, etc)